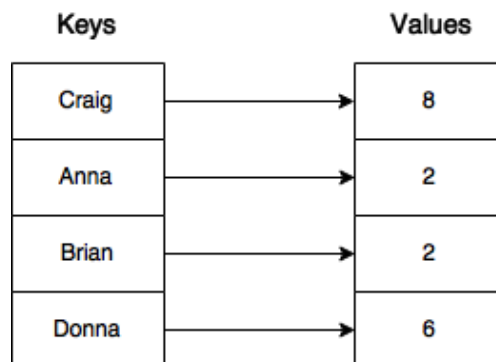


Chapter 9: Maps & Sets

By Irina Galata

A map is an unordered collection of pairs, where each pair is comprised of a **key** and a **value**.



As shown in the diagram above, keys are unique. The same key can't appear twice in a map, but different keys may point to the same value. All keys have to be of the same type, and all values have to be of the same type.

Maps are useful when you want to look up values by means of an identifier. For example, the table of contents of this book maps chapter names to their page numbers, making it easy to skip to the chapter you want to read.

How is this different from an array? With an array, you can only fetch a value by its index, which has to be an integer, and all indexes have to be sequential. In a map, the keys can be of any type and are generally in no particular order.

Creating maps

The easiest way to create a map is by using the standard library `mapOf()` function. This function takes a list of Kotlin `Pair` objects separated by commas:

```
var yearOfBirth = mapOf(  
    "Anna" to 1990,  
    "Brian" to 1991,  
    "Craig" to 1992,  
    "Donna" to 1993  
)
```

The Kotlin `Pair` objects are created using the infix `to` function. Note that `Map<K, V>` is an *interface*, which you'll learn more about later on. The concrete type that is created depends on which standard library function is called. The `mapOf()` function returns an immutable map of fixed size.

For your card game from an earlier chapter, instead of using the two arrays to map players to their scores, you can use a map:

```
var namesAndScores = mutableMapOf(  
    "Anna" to 2,  
    "Brian" to 2,  
    "Craig" to 8,  
    "Donna" to 6  
)  
println(namesAndScores) // > {Anna=2, Brian=2, Craig=8, Donna=6}
```

In this example, the type of the map is inferred to be `MutableMap<String, Int>`. This means `namesAndScores` is a map with strings as keys and integers as values, that is, a map from strings to integers.

When you print the map, you see there's generally no particular order to the pairs. Remember that, unlike arrays, maps are not guaranteed to be ordered!

You can pass in no arguments to the standard library functions to create an empty map like so:

```
namesAndScores = mutableMapOf()
```

...or create a new empty map by calling a map constructor:

```
var pairs = HashMap<String, Int>()
```

Specifying the type on the variable is not required here, since the compiler can infer the type from the constructor being called.

Note: If you're following along, you'll need to add the import, `import java.util.HashMap`, to the top of the file to use `HashMap`.

When you create a map, you can define its capacity:

```
pairs = HashMap<String, Int>(20)
```

This is an easy way to improve performance when you have an idea of how much data the map needs to store.

Accessing values

As with arrays, there are several ways to access map values.

Using the index operator

Maps support using square brackets to access values. Unlike arrays, you don't access a value by its index but rather by its key. For example, if you want to get Anna's score, you would type:

```
namesAndScores = mutableMapOf(
    "Anna" to 2,
    "Brian" to 2,
    "Craig" to 8,
    "Donna" to 6
)
// Restore the values

println(namesAndScores["Anna"])
// > 2
```

The map will check if there's a pair with the key Anna, and if there is, return its value. If the map doesn't find the key, it will return `null`.

```
namesAndScores["Greg"] // null
```

With arrays, out-of-bounds index access causes a runtime error, but not so for maps.

Index access on maps by the key is really powerful. You can find out if a specific player is in the game without having to iterate over all the keys, as you must do when you use an array.

Using properties and methods

In addition to using indexing, you can also use the `get()` function to access a value:

```
println(namesAndScores.get("Craig"))  
// > 8
```

In fact, using the index for a map is translated to a call the `get()` operator function.

Maps share many of the same properties and methods of other collection types. For example, both arrays and maps have `isEmpty()` and `size` members:

```
namesAndScores.isEmpty() // false  
namesAndScores.size      // 4
```

Modifying mutable maps

It's easy enough to create maps and access their contents — but what about modifying them? You'll need a mutable map to do so.

Adding pairs

Bob wants to join the game. Take a look at his details before you let him join:

```
val bobData = mutableMapOf(  
    "name" to "Bob",  
    "profession" to "CardPlayer",  
    "country" to "USA"  
)
```

This map is of type `MutableMap<String, String>`. Imagine you received more information about Bob and you wanted to add it to the map. This is how you'd do it:

```
bobData.put("state", "CA")
```

There's even a shorter way to add pairs, using subscripting:

```
bobData["city"] = "San Francisco"
```

Bob's a professional card player. So far, he sounds like a good addition to your roster.

Mini-exercise

Write a function that prints a given player's city and state.

Updating values

It appears that in the past, Bob was caught cheating when playing cards. He's not just a professional — he's a card shark! He asks you to change his name and profession so no one will recognize him.

Because Bob seems eager to change his ways, you agree. First, you change his name from Bob to Bobby:

```
bobData.put("name", "Bobby") // Bob
```

You saw this method above when you read about adding pairs. Why does it return the string Bob? `put(key: K, value: V): V?` replaces the value of the given key with the new value and returns the old value. If the key doesn't exist, this method will add a new pair and return null.

As with adding, you can do this with less code by using subscripting:

```
bobData["profession"] = "Mailman"
```

Like `put()`, this code updates the value for this key or, if the key doesn't exist, creates a new pair.

You can also use the `+=` infix operator to add a pair:

```
val pair = "nickname" to "Bobby D"
bobData += pair

println(bobData)
// > {name=Bobby, profession=Mailman, country=USA, state=CA,
city=San Francisco, nickname=Bobby D}
```

Removing pairs

Bob — er, sorry — *Bobby*, still doesn't feel safe, and he wants you to remove all information about his whereabouts:

```
bobData.remove("city")
bobData.remove("state", "CA")
```

This first call to `remove()` will remove the key `city` and its associated value from the map. The second call will remove the key only if the value matches the second argument.

Iterating through maps

The `for-in` loop works when you want to iterate over a map. But since the items in a map are pairs, you need to use a destructuring declaration:

```
for ((player, score) in namesAndScores) {
    println("$player - $score")
}
// > Anna - 2
// > Brian - 2
// > Craig - 8
// > Donna - 6
```

It's also possible to iterate over just the keys:

```
for (player in namesAndScores.keys) {
    print("$player, ") // no newline
}
println() // print a newline
// > Anna, Brian, Craig, Donna,
```

You can iterate over just the values in the same manner with the `values` property of the map.

Running time for map operations

In order to be able to examine how maps work, you need to understand what **hashing** is and how it works. Hashing is the process of transforming a value — `String`, `Int`, `Double`, `Boolean`, etc — to a numeric value, known as the *hash value*. This value can then be used to quickly look up the values in a *hash table*.

The Kotlin Any type defines a `hashCode()` method that will return a hash value for any object. All basic types already have a hash value. Here's an example:

```
println("some string".hashCode())  
// > 1395333309  
  
println(1.hashCode())  
// > 1  
println(false.hashCode())  
// > 1237
```

The hash value has to be deterministic — meaning that a given value must *always* return the same hash value. No matter how many times you calculate the hash value for `some string`, it will always give the same value.

You should never save a hash value, however, as there is no guarantee it will be the same from run-to-run of your program. Here's the performance of various hash map operations. This great performance hinges on having a good hashing function that avoids value collisions. If you have a poor hashing function, all of the operations below degenerate to linear time, or $O(n)$ performance. Fortunately, the built-in types have great, general purpose `hashCode()` implementations.

Accessing elements: Getting the value for a key is a constant time operation, or $O(1)$.

Inserting elements: To insert an element, the map needs to calculate the hash value of the key and then store data based on that hash. These are all $O(1)$ operations.

Deleting elements: Again, the map needs to calculate the hash value to know exactly where to find the element, and then remove it. This is also an $O(1)$ operation.

Searching for an element: As mentioned above, accessing an element has constant running time, so the complexity for searching is also $O(1)$.

While all of these running times compare favorably to arrays, remember that you generally lose order information when using maps.

For performance-critical code, `HashMap<K, V>` should be used via `hashMapOf()`, instead of `mapOf()`.

Sets

A set is an unordered collection of unique values of the same type. This can be extremely useful when you want to ensure that an item doesn't appear more than once in your collection, and when the order of your items isn't important.

Creating sets

You can declare a set explicitly by using the standard library `setOf()` function:

```
val names = setOf("Anna", "Brian", "Craig", "Anna")
println(names)
// > [Anna, Brian, Craig]
```

Notice in the above example that the duplicate "Anna" is not included in the set.

You can create an empty set by calling a constructor:

```
val hashSet = HashSet<Int>()
```

Set from arrays

Sets can be created from arrays. Consider this example:

```
val someArray = arrayOf(1, 2, 3, 1)
```

You can create a set from this array by passing the array into a standard library set function and using the **spread operator**:

```
var someSet = mutableSetOf(*someArray)
```

The array is spread into its elements when creating the set. You don't have to explicitly declare the variable as a `MutableSet<Int>`, since the type is inferred from the argument passed into the function.

To see the most important feature of a set in action, print the set you just created:

```
println(someSet) // > [1, 2, 3]
```

Although you created the set with two instances of the value 1, that value only appears once. Remember, a set's values must be unique.

Accessing elements

You can use `contains()` to check for the existence of a specific element:

```
println(someSet.contains(1))  
// > true
```

You can also use the `in` to check for existence:

```
println(4 in someSet)  
// > false
```

You can also use the `first()` and `last()` methods, which return one of the elements in the set. However, because sets are unordered, you won't always know exactly which item you'll get.

Finally, you can iterate through a set using a `for` loop the same way as you do for other collections:

```
for (number in someSet) {  
    println(number)  
}
```

Adding and removing elements

You can use `add()` to add elements to a set. If the element already exists, the method does nothing.

```
someSet.add(5)
```

You can remove the element from the set like this:

```
val removedOne = someSet.remove(1)  
println(removedOne) // > true  
  
println(someSet)  
// > [2, 3, 5]
```

`remove()` returns `true` if the element was removed from the set, or `false` otherwise.

Running time for set operations

Sets have a very similar implementations to those of maps, and they also require the elements to have hash values. The `HashSet` running time of all the operations is identical to those of a `HashMap`.

Challenges

Check out the following challenges to test your knowledge of maps and sets.

1. Which of the following are valid statements?

```
1. val map1: Map<Int to Int> = emptyMap()  
2. val map2 = emptyMap()  
3. val map3: Map<Int, Int> = emptyMap()
```

For the next four statements, use the following map:

```
val map4 = mapOf("One" to 1, "Two" to 2, "Three" to 3)
```

```
4. map4[1]  
5. map4["One"]  
6. map4["Zero"] = 0  
7. map4[0] = "Zero"
```

For the next three statements, use the following map:

```
val map5 = mutableMapOf(  
    "NY" to "New York",  
    "CA" to "California"  
)
```

```
8. map5["NY"]  
9. map5["WA"] = "Washington"  
10. map5["CA"] = null
```

2. Given a map with two-letter state codes as keys, and the full state names as values, write a function that prints all the states with names longer than eight characters. For example, for the map `mapOf("NY" to "New York", "CA" to "California")`, the output would be California.

3. Write a function that combines two maps into one. If a certain key appears in both maps, ignore the pair from the first maps. This is the function's signature:

```
fun mergeMaps(  
    map1: Map<String, String>,  
    map2: Map<String, String>  
) : Map<String, String>
```

4. Declare a function `occurrencesOfCharacters` that calculates which characters occur in a string, as well as how often each of these characters occur. Return the result as a map. This is the function signature:

```
fun occurrencesOfCharacters(text: String): Map<Char, Int>
```

Hint: `String` is a collection of characters that you can iterate over with a `for` statement.

Bonus: To make your code shorter, maps have a special function that lets you add a default value if it is not found in the map. For example, `map.getOrElse('a', defaultValue = 0)` returns 0 for the character 'a' if it is not found, instead of simply returning `null`.

5. Write a function that returns `true` if all of the values of a map are unique. Use a set to test uniqueness. This is the function signature:

```
fun isInvertible(map: Map<String, Int>): Boolean
```

6. Given the map:

```
val nameTitleLookup: Map<String, String?> = mutableMapOf(  
    "Mary" to "Engineer",  
    "Patrick" to "Intern",  
    "Ray" to "Hacker"  
)
```

Set the value of the key "Patrick" to `null` and completely remove the key and value for "Ray".

Key points

- A **map** is an unordered collection of key-value pairs.
- The **keys** of a map are all of the same type, and the **values** are all of the same type.
- Use **indexing** to get values and to add, update or remove pairs.
- If a key is not in a map, lookup returns `null`.
- Built-in Kotlin types such as `String`, `Int`, `Double` have efficient hash values out of the box.
- Use `HashMap<K, V>` for performance critical code.
- **Sets** are unordered collections of unique values of the same type.
- Sets are most useful when you need to know whether something is included in the collection or not.

Where to go from here?

Now that you've learned about collection types in Kotlin, you should have a good idea of what they can do and when you should use them. You'll see them come up as you continue on in the book.

The next chapter of the book covers **lambdas**. One of the many great features of lambdas is that they let you iterate over the collection types you've learned in a less explicit and more readable manner than loops.